

# FRUIT CATCHER REPAIR LAB

## Debugging and Implementation Record

**NAME : D R Rushmitha**

**SRN : PES1UG24AM910**

**SECTION : E**

### 1. Original Bug

#### USER PROMPT

Please inspect the repository and understand the existing Pygame project before suggesting changes. The lab requires me to fix the deliberate floor-miss scoring/life deduction bug, add rotten fruit / hazard bombs, add dynamic falling-speed and spawn-rate escalation as the score increases, and add fruit splash particle effects. First, identify the original bug and explain what is currently happening in the code. Do not implement the features yet. Also tell me what I should demonstrate in my BEFORE gameplay video.

#### RESPONSE

The original defect was located in game/game\_engine.py. The basket collision branch correctly awarded one point, but the floor-miss branch also awarded one point instead of deducting a life. Lives were initialized to 3 but were not reduced when a fruit was missed.

```
if fruit.is_missed(self.height):  
    self.score += 1  
    self.fruits.remove(fruit)
```

Therefore, the original behavior was: catching a normal fruit increased the score, but missing a normal fruit also increased the score. A missed fruit did not reduce lives.

The BEFORE video was intended to show a fruit missing the basket, reaching the floor, disappearing, and incorrectly increasing the score.

### 2. Task 1 — Bug Fix

#### USER PROMPT

Task 1 has been fixed and tested successfully. The intended behavior is: Catch normal fruit → +1 score; Miss normal fruit → -1 life; Missing a fruit must not increase the score; At 0 lives → GAME OVER; Pressing R after Game Over → restart with score 0 and lives 3. Identify the exact files and code sections that need to be changed and provide the corrected implementation. Do not implement Tasks 2–4 yet.

#### RESPONSE

Only game/game\_engine.py needed to be changed for Task 1. The catch branch remained responsible for

increasing the score. The miss branch was changed to reduce lives and enter GAME\_OVER when lives reached zero.

```
if basket_rect.colliderect(fruit.rect):
    self.score += 1
    self.fruits.remove(fruit)
    continue

if fruit.is_missed(self.height):
    self.lives -= 1
    self.fruits.remove(fruit)

    if self.lives <= 0:
        self.game_state = "GAME_OVER"
```

The existing reset mechanism was retained. It restores the game to score 0, lives 3, clears the fruits, and returns the game state to PLAYING.

Testing result: the user explicitly reported that Task 1 was fixed and tested successfully.

### 3. Tasks 2–4 — Implementation

#### USER PROMPT

**Task 1 has been fixed and tested successfully. Now help me implement the remaining three tasks while preserving the existing project structure: Task 2 — Hazard bombs; Task 3 — Dynamic difficulty; Task 4 — Splash particles. Avoid unnecessary rewrites.**

#### RESPONSE

The remaining features were designed to preserve the existing project structure. Changes were concentrated in game/fruit.py and game/game\_engine.py, with a small new game/particle.py class. main.py did not need to be rewritten.

#### **Task 2 — Hazard Bombs**

A hazard flag was added to falling objects. A small percentage of spawned objects are hazards, and hazards are rendered differently from normal fruits.

```
is_hazard = random.random() < 0.10

self.fruits.append(
    Fruit(
        self.width,
        speed_multiplier=speed_multiplier,
        is_hazard=is_hazard,
    )
)
```

The implemented rules are: catching a hazard deducts one life without increasing the score; missing a hazard has no penalty. Normal fruit behavior remains unchanged: catching gives +1 score and missing deducts one life.

### Task 3 — Dynamic Difficulty

Difficulty was made dependent on score. As score increases, the spawn delay decreases and the falling-speed multiplier increases. Limits were included so the game remains playable.

```
def get_difficulty(self):
    progress = min(self.score / self.difficulty_score, 1.0)

    spawn_delay = (
        750
        - (750 - self.min_spawn_delay) * progress
    )

    speed_multiplier = (
        1.0
        + (self.max_speed_multiplier - 1.0) * progress
    )

    return spawn_delay, speed_multiplier
```

The implementation used a starting spawn delay of 750 ms, a minimum of 250 ms, a starting speed multiplier of 1.0×, and a maximum speed multiplier of 1.75×.

### Task 4 — Splash Particles

A new game/particle.py class was used for lightweight splash particles. Each particle has a position, velocity, color, radius, and short lifetime. Particles move outward, are affected by simple gravity, and are removed when their lifetime expires.

```
class SplashParticle:
    def update(self):
        self.x += self.vx
        self.y += self.vy
        self.vy += 0.15
        self.age += 1

    def is_alive(self):
        return self.age < self.lifetime
```

The engine creates colored splash particles when a normal fruit is caught and when a normal fruit reaches the ground. Particles are updated independently and disappear after a short time.

## 4. AFTER Video Checklist

### USER PROMPT

**I have implemented Tasks 1–4 and tested the updated game. I am now recording my AFTER gameplay video. Please give me a concise checklist of behaviors that I should demonstrate so that it provides evidence that the implementation works. The checklist should cover correct scoring, life deduction, hazard behavior, dynamic difficulty, splash particles, Game Over, and restart functionality. After I finish the video, I will report what I actually observed. Do not assume a feature worked unless I confirm it.**

### RESPONSE

The recommended evidence was to demonstrate: normal fruit caught = +1 score; normal fruit missed = -1 life with no score increase; hazard caught = -1 life; hazard missed = no penalty; increasing score = faster

falling and more frequent spawning; normal fruit caught = splash; normal fruit reaching the ground means splash; lives reaching zero means GAME OVER; and pressing R means restart.

## 5. Reported AFTER Observation

### USER PROMPT

**Now in the AFTER video I have made changes. When the lives get over it is asking me to restart, so hence the changes have been made.**

### RESPONSE

This confirms that the updated game reached the Game Over state and displayed the restart prompt when the lives were exhausted. The reset values should only be described as experimentally verified if they were actually observed after pressing R.

## 6. Final Review

### USER PROMPT

**I have now recorded both gameplay videos: BEFORE shows the original floor-miss bug where missing a fruit incorrectly increases the score; AFTER shows the repaired game and newly implemented features. Please perform a final review of the repository and summarize the complete debugging process. Compare original and final behavior and explain: (1) original bug, (2) how it was fixed, (3) hazard mechanic, (4) dynamic difficulty, (5) splash particles, (6) Game Over and restart, and (7) what evidence should be visible in BEFORE and AFTER videos. Only state behaviors supported by the repository or by the testing results I provided. Do not claim something was experimentally verified if I did not report testing it.**

### RESPONSE

The original bug was the floor-miss branch awarding +1 score instead of reducing lives. Task 1 corrected the miss handling and added the Game Over transition at zero lives. Hazards were implemented as a small percentage of visually distinct falling objects; catching a hazard costs one life, while missing one carries no penalty. Dynamic difficulty was implemented by reducing spawn delay and increasing the falling-speed multiplier as score increases, with practical limits. Splash particles were implemented for normal-fruit catches and normal-fruit ground impacts, with short lifetimes. Game Over occurs when lives reach zero, and the restart path restores the initial game state.

Evidence distinction: the BEFORE floor-miss bug was explicitly reported as visible in the recording. Task 1 was explicitly reported as fixed and tested successfully. The Game Over/restart prompt was explicitly observed in the AFTER video. The individual hazard, dynamic-difficulty, and splash behaviors were described as implemented, but were not separately reported as experimentally verified in the conversation.